

# Learning and reusing primitive behaviours to improve Hindsight Experience Replay sample efficiency

Francisco Roldan Sanchez  
 Dublin City University  
 Insight SFI Centre for Data Analytics  
 Dublin, Ireland  
 francisco.sanchez@insight-centre.org

Qiang Wang  
 University College Dublin  
 Dublin, Ireland  
 qiang.wang@ucdconnect.ie

David Cordova Bulens  
 University College Dublin  
 Dublin, Ireland  
 david.cordovabulens@ucd.ie

Kevin McGuinness  
 Dublin City University  
 Insight SFI Centre for Data Analytics  
 Dublin, Ireland  
 kevin.mcguinness@insight-centre.org

Stephen J. Redmond  
 University College Dublin  
 Insight SFI Centre for Data Analytics  
 Dublin, Ireland  
 stephen.redmond@ucd.ie

Noel E. O'Connor  
 Dublin City University  
 Insight SFI Centre for Data Analytics  
 Dublin, Ireland  
 noel.oconnor@insight-centre.org

**Abstract**—Hindsight Experience Replay (HER) is a technique used in reinforcement learning (RL) that has proven to be very efficient for training off-policy RL-based agents to solve goal-based robotic manipulation tasks using sparse rewards. Even though HER improves the sample efficiency of RL-based agents by learning from mistakes made in past experiences, it does not provide any guidance while exploring the environment. This leads to very large training times due to the volume of experience required to train an agent using this replay strategy. In this paper, we propose a method that uses primitive behaviours that have been previously learned to solve simple tasks in order to guide the agent toward more rewarding actions during exploration while learning other more complex tasks. This guidance, however, is not executed by a manually designed curriculum, but rather using a critic network to decide at each timestep whether or not to use the actions proposed by the previously-learned primitive policies. We evaluate our method by comparing its performance against HER and other more efficient variations of this algorithm in several block manipulation tasks. We demonstrate the agents can learn a successful policy faster when using our proposed method, both in terms of sample efficiency and computation time. Code is available at <https://github.com/franroldans/qmp-her>.

**Index Terms**—reinforcement learning, robotic manipulation, experience replay

## I. INTRODUCTION

Data-driven robotic manipulation has become very popular in recent years due to the success of reinforcement learning

This publication has emanated from research supported by Science Foundation Ireland (SFI) under Grant Number SFI/12/RC/2289\_P2, co-funded by the European Regional Development Fund, by Science Foundation Ireland Future Research Leaders Award (17/FRL/4832), and by China Scholarship Council (CSC No.202006540003). For the purpose of Open Access, the author has applied a CC BY public copyright licence to any Author Accepted Manuscript version arising from this submission. We would like to express our deepest gratitude to Prof. Kevin McGuinness for his valuable contributions to this research. Unfortunately, he passed away before the completion of this work. He will be remembered for his exceptional insights and dedication.

(RL) algorithms [1]. By analysing large amounts of data, data-driven robotic manipulation methods allow the robot to carry out tasks with higher precision and efficiency [2]. Furthermore, because these techniques let robots learn from successes and mistakes, they can potentially be used to continuously improve the robot's abilities over time, leading to greater adaptability to different situations without the need of specialised programming [3]. However, to achieve such levels of adaptability, it is usually necessary to generate large quantities of good quality data, which leads to prohibitively large training times for tasks and robots with complex observation, goal or/and action spaces. Therefore, training RL-based agents on such environments can be very challenging or even infeasible depending on the computational resources available [4]–[6].

The most popular techniques used to solve data-driven robotic manipulation tasks are off-policy reinforcement learning methods, due to their sample efficiency and their ability to decouple exploration from exploitation [7]–[9]. Firstly, off-policy reinforcement learning algorithms are more sample efficient because they update their data after analysing a diverse set of trajectories, making them converge faster than when being immediately updated after each interaction with the environment. Secondly, exploration and exploitation are decoupled, meaning that one can use a different policy for exploration, termed a *behaviour policy*, while the learning policy, termed a *target policy*, prioritises exploiting what is learned from these well-chosen actions. One popular technique is Deep Deterministic Policy Gradients (DDPG) [7] in conjunction with Hindsight Experience Replay (HER) [10], which has proven to be very efficient for goal-based tasks with sparse rewards (i.e. binary rewards).

This paper proposes a novel method that evaluates and selects behaviours generated by policies that are capable of

performing simpler primitive skills. In this way we can accelerate the learning of new more skillful policies able to perform more difficult manipulation tasks by integrating this method of suggesting and selecting actions using primitive policies into the exploration-exploitation logic of the learning algorithm. We evaluate our method on four different robotic block manipulation tasks, comparing its success rate, its sample efficiency, and its computation time against an agent trained using the original HER algorithm and other more efficient variants of HER.

## II. BACKGROUND

### A. Q-learning

Q-learning [11] is a popular off-policy RL algorithm used to train an agent to make optimal decisions in a Markov decision process environment. When training an agent using Q-learning, a Q-table of state-action pairs must be preset by an initial exploration, along with the state-action pair Q-value. The Q-value is a metric that captures the quality of the transition, which is represented by a state  $s$ , an action  $a$ , the future state  $s'$  reached by the taking the action, and a reward  $r$ . In other words, it defines how good or bad an action  $a$  is that is taken for a particular state  $s$ , when following a policy  $\pi$ .

In particular, the Q-value for a state is the expected discounted cumulative reward given the state and the action. During training, as the environment is further explored, these Q-values must be updated based on the observed rewards using the Bellman equation [12]:

$$Q_{new}^{\pi} = Q(s, a) + \alpha[R(s, a) + \gamma \max_{a'}(Q(s', a')) - Q(s, a)], \quad (1)$$

where  $Q(s, a)$  denotes for the current Q-value,  $\alpha$  is the learning rate,  $R(s, a)$  is the reward for taking action  $a$  when in state  $s$ ,  $\gamma$  is the discount factor, determining the importance of future rewards, and  $\max(Q(s', a'))$  is the maximum expected reward in the new state  $s'$  given all possible actions that can be taken in the next state  $s'$ .

### B. Deep Deterministic Policy Gradients and Hindsight Experience Replay

DDPG [7] is an off-policy RL algorithm used for continuous control that is built using an actor-critic architecture, with an actor network  $\pi(s)$  that predicts actions and a critic network  $Q^{\pi}(s, a)$  that estimates the associated Q-value (a measure of quality) of pairs of states and actions.

During training, the actor network updates its parameters to maximise the estimated Q-value, while the critic network is updated by minimising the mean squared temporal difference error [13] between the estimated Q-value and the target Q-value, which is computed using equation 1. In order to perform these updates, data is sampled from a replay buffer that stores a diverse set of trajectories obtained from previous interactions with the environment (i.e. temporal sequences of environment transitions as defined in section II-A).

HER [10] is a replay strategy that accelerates the learning of off-policy RL algorithms by altering the stored data from

past experiences. While in a standard RL framework an agent only can learn from trajectories receiving a positive reward, HER changes that by modifying the goals of failed stored experiences so that they match the achieved goal, thus obtaining a positive reward from what was a failed episode. In other words, it makes the agent learn from its own mistakes.

Despite the benefits of HER, this method requires a large quantity of simulated experience to find a good policy. The main reason for this requirement is the lack of guidance given to the agent during exploration [14], [15], particularly in the first interactions with the environment, as HER becomes most useful when the agent interacts with the object and changes its state, creating greater variety in the hindsight goals.

There are some algorithms that modify HER in order to tackle this problem. Hindsight Goal Generation (HGG) [14] addresses the issue by generating a set of alternative goals based on the achieved states during the exploratory episodes. Alternatively, in [15], a method named Graph-based Hindsight Goal Generation (G-HGG) is proposed to replace the Euclidean distance metric normally used to evaluate goal-based tasks by a graph-based distance metric, and in this way improve the selection of intermediate goals in environments with physical obstacles. However, both of these methods have very time-consuming policy updates, mostly because of the trajectory sampling required to use them, as for each generated goal, trajectories that match the generated goal states must be sampled from the replay buffer.

### C. Ensemble of policies

An ensemble of policies is a collection of  $K$  individual policies  $\mathbb{P} = \{\hat{\pi}_1, \hat{\pi}_2, \dots, \hat{\pi}_K\}$  that are used together to solve a task. Each policy in the ensemble  $\mathbb{P}$  may have different strengths and weaknesses, and by combining their decisions, the ensemble can benefit from the range and variety of skills possessed by the policies that make up the ensemble, this improving the overall performance of the agent.

There are different strategies to create such ensembles. For example, in [16], the authors define a hierarchical policy, with a high-level task policy that decides which lower-level primitive policy of the ensemble to use, and a low-level parameter policy that determines how to instantiate the chosen primitive, in order to learn how to order the selection of these policies and their parameters over time.

SUNRISE [17] is another ensemble mechanism that trains  $N$  independent actor-critic based agents in a task and then selects the candidate action of the agent that maximises a *weighted Bellman backup* equation, which is calculated using the mean and variance of the Q-values obtained by each candidate action using the critics of each of the agents of the ensemble. This idea of filtering actions based on the Q-value has also been used in different imitation learning frameworks and offline RL methods to filter noisy actions or unwanted behaviours [18], [19].

A more recent method is the Q-switch Mixture of Policies (QMP) [20], which uses a Q-filter in a multi-task setting designed to simultaneously find policies that solve different tasks

that share the observation space in order to identify shareable behaviours between policies. During exploration, each of the policies of the ensemble proposes a candidate action, but only the action that maximises the Q-value associated with the target policy for the task being solved is executed.

In this paper, we use a QMP to improve the sample efficiency of HER, but instead of sharing behaviours between policies that are being simultaneously trained in the same environment, we make use of fixed primitive policies that do not necessarily share the observation or/and the goal spaces with the target task.

### III. METHOD

The focus of this work is to improve the sample efficiency of DDPG agents trained with HER for robotic manipulation tasks by reusing simpler primitive skills that can be more easily learned. These primitive skills are policies obtained by training a DDPG agent using HER in simpler tasks.

We define a set of  $L$  objectives  $\mathbb{O} = \{o_1, o_2, \dots, o_L\}$  that correspond to potential state-goal combinations that  $K$  primitive policies  $\mathbb{P} = \{\hat{\pi}_1(s, g), \hat{\pi}_2(s, g), \dots, \hat{\pi}_K(s, g)\}$  could use. For each timestep during exploration, we use each primitive policy in  $\mathbb{P}$  to create a candidate action for each of the objectives, and store them along with the candidate action  $a_t^\pi$  that the target policy  $\pi(s, g)$  proposes. After that, all candidate actions  $\mathcal{A}_t = \{a_t^\pi, a_t^{1,1}, a_t^{1,2}, \dots, a_t^{K,L}\}$  for that timestep are evaluated by the critic of the target policy  $Q^\pi(s, a)$ , and only the action achieving the maximum Q-value for the current state  $s_t$  is executed and the transition stored, as done in [20]:

$$a^* = \arg \max Q^\pi(s_t, g, \mathcal{A}_t) \quad (2)$$

Before any update is performed, the Q-value that the critic predicts will be completely random, and as such during warm-up timesteps (i.e. timesteps of data collection before the actual learning process begins) the agent is forced to pick an action from the suggestions coming from the ensemble of primitive policies, along with the  $\epsilon$ -greedy algorithm (i.e., the agent occasionally explores the environment using random actions with a probability  $\epsilon$ ). After the warm-up, the agent also considers the actions suggested by the target policy.

For all tasks, it is expected that the actions suggested by the primitive policies will mostly be used during the first iterations of the exploration, but as the target policy continues to update, the actions predicted by the target policy will have a higher Q-value. This means that by the end of the training, our method's behaviour will naturally evolve to be the same as original HER.

We call this algorithm Q-switch Mixture of Primitives Hindsight Experience Replay (QMP-HER) - see Algorithm 1 for more details.

### IV. EXPERIMENTS

#### A. Environment

We make use of the block manipulation tasks proposed by OpenAI for the Fetch robot arm using a parallel-jaw

Algorithm 1: Q-switch Mixture of Primitives Hindsight Experience Replay

**Given:** Off-policy RL agent  $\mathbb{A} \sim \pi(s, g), Q^\pi(s, g, a)$ .  
 $K$  primitives:  $\mathbb{P} = \{\hat{\pi}_1(s, g), \hat{\pi}_2(s, g), \dots, \hat{\pi}_K(s, g)\}$

Replay Buffer  $\mathcal{R}$

```

1: Initialise  $\mathbb{A}$ 
2: Load  $\mathbb{P}$ 
3: Define a set of  $L$  objectives  $\mathbb{O} = \{o_1, o_2, \dots, o_L\}$  for  $\mathbb{P}$ 
4: for epoch = 0 to  $N$  do
5:   for episode = 1 to  $M$  do
6:     Sample a goal  $g$  and an initial state  $s_0$ 
7:     for  $t = 0$  to  $T - 1$  do
8:       for  $k = 1$  to  $K$  do
9:         for  $l = 1$  to  $L$  do
10:           $s', g' \sim$  Adjust state and/or goal based on  $o_l$ 
11:          Sample candidate action  $a_t^{k,l}$  from  $\hat{\pi}_k(s', g')$ 
12:           $\mathcal{A}_t \sim$  Store candidate action  $a_t^{k,l}$ 
13:        end for
14:      end for
15:      Sample candidate action  $a_t^\pi$  from  $\pi(s_t, g)$ 
16:       $\mathcal{A}_t \sim$  Store candidate action  $a_t^\pi$ 
17:      Select best action  $a^*$  using Eq. 2, applying  $\epsilon$ -greedy
18:      Execute action  $a^*$  and observe state  $s_{t+1}$ 
19:       $\mathcal{R} \sim$  Store transition  $(a^*, s_t, s_{t+1}, g, r_t)$ 
20:    end for
21:  end for
22:  Apply HER logic and update  $\pi(s, g), Q^\pi(s, g, a)$ 
23: end for

```

TABLE I: Overview of the different environments observation and goal spaces

| Environment              | Obs. Dimension | Object info. | Obstacles | Goal space |
|--------------------------|----------------|--------------|-----------|------------|
| <i>FetchReach</i>        | 10             | ✗            | ✗         | Continuous |
| <i>FetchPush</i>         | 25             | ✓            | ✗         | Continuous |
| <i>FetchPickAndPlace</i> | 25             | ✓            | ✗         | Continuous |
| <i>FetchPickObstacle</i> | 25             | ✓            | ✓         | Continuous |
| <i>FetchPickAndThrow</i> | 25             | ✓            | ✓         | Discrete   |

gripper [4], as well as two more tasks proposed in [15]. These are summarised as follows in order of complexity:

- *FetchReach*: the goal is to move the parallel-jaw gripper to a position in the 3D space. The gripper remains closed (Fig 1a).
- *FetchPush*: the goal is to move a block to a point in the 3D space that is over the table. The gripper remains closed (Fig 1b).
- *FetchPickAndPlace*: the goal is to move a block to a point in the 3D space. Gripper controls are activated (Fig 1c).
- *FetchPickObstacle*: the goal is to move a block to a point in the 3D space with an obstacle between the initial and the target position. Gripper controls are activated (Fig 1d).
- *FetchPickAndThrow*: the goal is to lift a cube and throw it into one of eight boxes that are out of reach for the robotic arm. Gripper controls are activated (Fig 1e).

To improve sample efficiency, we aim to make use of the

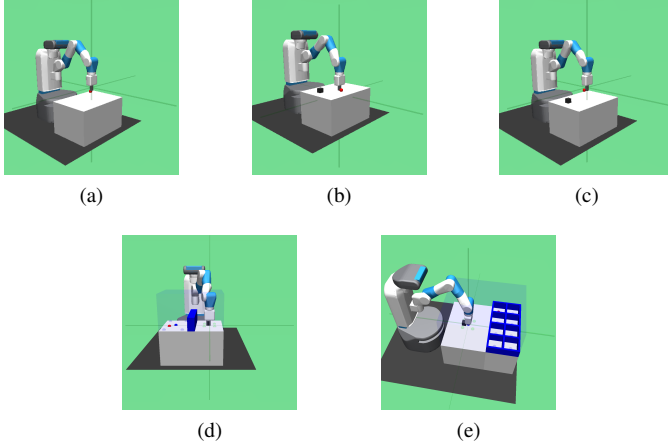


Fig. 1: Visual representation of (a) *FetchReach*, (b) *FetchPush*, (c) *FetchPickAndPlace*, (d) *FetchPickObstacle* and (e) *FetchPickAndThrow* tasks.

behaviour learned by the agents trained on the simplest tasks. Therefore, the method we present in this paper cannot be used in the *FetchReach* environment, as it is the simplest of all and it must be trained with HER. Once this policy is learned, there is then knowledge of the dynamics of the robot that has been acquired by this behaviour policy which can be used to more quickly learn how to perform other more complex tasks.

Because the observation space of the *FetchReach* environment is smaller than the observation space of the other three environments, it is necessary to remove the object state information from the observations of these other environments in order to use the policy learned on the *FetchReach* task.

### B. Experimental setup

Our method requires a heuristic definition of the objectives for the primitive policies. These objectives are defined based on regions of interest in the 3D space relative to the object and the goal position, and might vary depending on the target task. For example, in order to learn a push or lifting policy, a robot can benefit from approaching the object from all possible directions. However, when learning a policy in tasks with physical obstacles, the robot benefits from manipulating the object in an elevated position, avoiding the obstacles.

To solve the *FetchPush* task we define the objectives by setting a number of different goals for the *FetchReach* primitives. For each observation of the *FetchPush* task, we adjust it by removing the object information and request the *FetchReach* primitive policy to move towards the location of the object and the goal position, but we also include objectives that point toward the object with slight deviations, so that the gripper touches the object from the left, right, front and behind. To solve the *FetchPickAndPlace* task, we make use of the same primitives and objectives as the used to solve the *FetchPush* task. For this experiment, we also use the target policy during warm-up timesteps as it is necessary for

the gripper state exploration because the policy trained on the *FetchReach* task does not have the gripper controls activated.

The *FetchPickObstacle* is a task of sufficient complexity that HER cannot solve it. However, the task has many similarities with the *FetchPickAndPlace* task and the policy learned in that task can be useful to accelerate the learning pace. We perform two different experiments: the first one being a manually designed curriculum which we implement by enforcing the behaviour policy to perform the first 20 timesteps of each episode using the policy learned on the *FetchPickAndPlace* task with the goal to lift the object over the obstacle, the following 20 timesteps to move it towards the original goal, and for the remainder of the episode the agent executes the actions predicted by its actor network  $\pi(s)$ . We call this framework *PickPlace&HER*. In the second experiment, we use our method defining two objectives for the *FetchPickAndPlace* primitive, one that aims to pick the block and move it above the obstacle and a second one aiming for the goal position.

The *FetchPickAndThrow* task is much more challenging than the previous tasks, as the robot has to lift up the cube and throw it into a basket, meaning that the goal space is a discrete set of eight regions instead of the continuous goal space we had in previous experiments. Similar to the *FetchPickObstacle* task, we believe that this task can be accelerated by starting with the object grasped by the gripper. Therefore, we run two different experiments, a first one running the same strategy as we defined for the *PickPlace&HER* exploration algorithm (i.e., grasping the object and moving it to a centred and elevated position, then letting the target policy explore). The second experiment involves our method using a combination of primitive behaviours: we use the policy learned on the *FetchPickObstacle* task, setting as the objective the same goal that was sampled for that episode; and the policy learned on the *FetchPickAndPlace* task, setting as objectives a centred and elevated position and the sampled goal for that episode.

To calculate the success rate, we do not apply the QMP-HER logic (behaviour policy). Instead, we only test the actions predicted by the actor of the target policy  $\pi(s)$  that is being updated to learn the execution of the task end-to-end.

## V. RESULTS

The results demonstrate that our method was able to accelerate the learning of HER, as it is capable of reaching a convergence point faster than HER (Fig 2).

In the case of the *FetchPush* task, our method is able to obtain a success rate of 80% in less than 20 policy updates, while it takes about 60 policy updates for HER to achieve the same performance. Our method is also able to achieve a 90% success rate faster, as it only needs 25 policy updates, while HER requires approximately 70 updates (see Fig 2a). A similar result is achieved for the *FetchPickAndPlace* task, where the success rate of our method reaches 90% accuracy using approximately 150 policy updates while HER takes about 180 policy updates, proving that our method increases the sample efficiency (see Fig 2b).

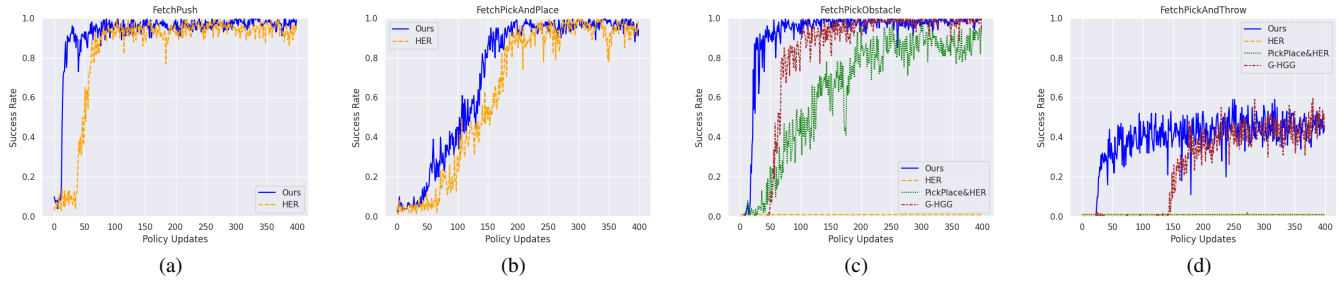


Fig. 2: Success rate of the target policy  $\pi(s, g)$  per each policy update on the different experiments described in Section IV.

The results obtained for the different experiments for the *FetchPickObstacle* task can be seen in Fig 2c. While the task is too difficult for HER, which completely fails, our approach was able to reach an average of 95% success rate in about 60 policy updates, compared to the approximate 175 updates needed for G-HGG. Furthermore, the average time per policy update of our model is 86 seconds, while for G-HGG it takes 117 seconds. The *PickPlace&HER* behaviour policy, while still working better than HER, proved to be inferior to the other methods explored, as the learning pace is slower.

For the *FetchPickAndThrow* experiments, we found that our method and G-HGG are able to achieve a 50% success rate, while the other methods are unable to learn any useful policy (see Fig 2d). After analysing the behaviour of the learned policy, it can be seen that for both methods the robot is observed to be capable throwing the block to the buckets of the first row nearest the robot base, but always fails when aiming for the second row buckets that are further away. This happens because during exploration the block almost never ends up in one of the second row buckets, making it impossible to generate hindsight goals for HER in these cases. Despite the similar results in terms of performance, our method is more sample efficient, as it converges faster than G-HGG, and more significantly, for each policy update our method takes 72.8 seconds on average, while the average time per update when using G-HGG is 396.5 seconds.

## VI. CONCLUSION

We have presented a novel algorithm, which we called Q-switch Mixture of Primitives Hindsight Experience Replay (QMP-HER), that improves the sample efficiency and the computation time of agents trained with HER by making use of primitive skills (i.e., policies) learned in simpler tasks. Instead of enforcing a curriculum for the behaviour policy using these primitives, the critic network of the target policy decides at each timestep during the exploration phase if the actions suggested by the primitive policies might be useful or not to solve the target task.

Future work arising from this publication could include finding better primitives that replicate more specific behaviours, possibly by clustering environment transitions of simple tasks and replicating the behaviour of these clusters using any state-

of-the-art behavioural modelling technique, such as decision transformers [21] or trajectory transformers [22].

One of the weaknesses of the algorithm presented in this paper is the need for manually defined objectives and, therefore, another future direction could explore employing our method in conjunction with HGG, G-HGG as well as other algorithms that propose intermediate goals during the exploration phase, or even introducing a low-level parameter policy that determines how to instantiate the primitives given an environment state as in [16].

## REFERENCES

- [1] Nguyen, Hai, and Hung La. "Review of deep reinforcement learning for robot manipulation." In 2019 Third IEEE International Conference on Robotic Computing (IRC), pp. 590-595. IEEE, 2019.
- [2] Imtiaz, M., Yuansong Qiao, and Brian Lee. "Comparison of Two Reinforcement Learning Algorithms for Robotic Pick and Place with Non-Visual Sensing." *Int. J. Mech. Eng. Robot. Res* 10, no. 10 (2021): 526-535.
- [3] Manh, Tien Ngo, Cuong Nguyen Manh, Dung Pham Tien, Manh Tran Van, Duyen Ha Thi Kim, and Duy Nguyen Duc. "Autonomous navigation for omnidirectional robot based on deep reinforcement learning." *International Journal of Mechanical Engineering and Robotics Research* 9, no. 8 (2020): 1134-1139.
- [4] Plappert, Matthias, Marcin Andrychowicz, Alex Ray, Bob McGrew, Bowen Baker, Glenn Powell, Jonas Schneider. "Multi-goal reinforcement learning: Challenging robotics environments and request for research." *arXiv preprint arXiv:1802.09464* (2018).
- [5] Melnik, Andrew, Luca Lach, Matthias Plappert, Timo Korthals, Robert Haschke, and Helge Ritter. "Using tactile sensing to improve the sample efficiency and performance of deep deterministic policy gradients for simulated in-hand manipulation tasks." *Frontiers in Robotics and AI* 8 (2021): 538773.
- [6] Sanchez, Francisco Roldan, Qiang Wang, David Cordova Bulens, Kevin McGuinness, Stephen J. Redmond, and Noel E. O'Connor. "Hierarchical reinforcement learning for in-hand robotic manipulation using Davenport chained rotations." In 2023 9th International Conference on Automation, Robotics and Applications (ICARA), pp. 160-164. IEEE, 2023.
- [7] Lillicrap, Timothy P., Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. "Continuous control with deep reinforcement learning." *arXiv preprint arXiv:1509.02971* (2015).
- [8] Schaul, Tom, John Quan, Ioannis Antonoglou, and David Silver. "Prioritized experience replay." *arXiv preprint arXiv:1511.05952* (2015).
- [9] Zha, Daochen, Kwei-Heng Lai, Kaixiong Zhou, and Xia Hu. "Experience replay optimization." *arXiv preprint arXiv:1906.08387* (2019).
- [10] Andrychowicz, Marcin, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. "Hindsight experience replay." *Advances in Neural Information Processing Systems* 30 (2017).
- [11] Watkins, Christopher JCH, and Peter Dayan. "Q-learning." *Machine Learning* 8 (1992): 279-292.

- [12] Dolcetta, I. Capuzzo, and Hitoshi Ishii. "Approximate solutions of the Bellman equation of deterministic control theory." *Applied Mathematics and Optimization* 11, no. 1 (1984): 161-181.
- [13] Tesauro, Gerald. "Temporal difference learning and TD-Gammon." *Communications of the ACM* 38, no. 3 (1995): 58-68.
- [14] Ren, Zhizhou, Kefan Dong, Yuan Zhou, Qiang Liu, and Jian Peng. "Exploration via hindsight goal generation." *Advances in Neural Information Processing Systems* 32 (2019).
- [15] Bing, Zhenshan, Matthias Brucker, Fabrice O. Morin, Rui Li, Xiaojie Su, Kai Huang, and Alois Knoll. "Complex robotic manipulation via graph-based hindsight goal generation." *IEEE transactions on neural networks and learning systems* 33, no. 12 (2021): 7863-7876.
- [16] Nasiriany, Soroush, Huihan Liu, and Yuke Zhu. "Augmenting reinforcement learning with behavior primitives for diverse manipulation tasks." In *2022 International Conference on Robotics and Automation (ICRA)*, pp. 7477-7484. IEEE, 2022.
- [17] Lee, Kimin, Michael Laskin, Aravind Srinivas, and Pieter Abbeel. "Sunrise: A simple unified framework for ensemble learning in deep reinforcement learning." In *International Conference on Machine Learning*, pp. 6131-6141. PMLR, 2021.
- [18] Yu, Tianhe, Aviral Kumar, Yevgen Chebotar, Karol Hausman, Sergey Levine, and Chelsea Finn. "Conservative data sharing for multi-task offline reinforcement learning." *Advances in Neural Information Processing Systems* 34 (2021): 11501-11516.
- [19] Sasaki, Fumihiro, and Ryota Yamashina. "Behavioral cloning from noisy demonstrations." In *International Conference on Learning Representations*. 2020.
- [20] Zhang, Grace, Ayush Jain, Injune Hwang, Shao-Hua Sun, and Joseph J. Lim. "Efficient Multi-Task Reinforcement Learning via Selective Behavior Sharing." *arXiv preprint arXiv:2302.00671* (2023).
- [21] Chen, Lili, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. "Decision transformer: Reinforcement learning via sequence modeling." *Advances in neural information processing systems* 34 (2021): 15084-15097.
- [22] Janner, Michael, Qiyang Li, and Sergey Levine. "Offline reinforcement learning as one big sequence modeling problem." *Advances in neural information processing systems* 34 (2021): 1273-1286.